

Answer Key for
PostgreSQL Essential – 16





Table of Contents

Module 3 - PostgreSQL Installation	4
Lab Exercise - 1	4
Answer Lab Exercise 1 (RHEL/Rocky/OL Linux 9)	4
Module 4 - User Tools - Command Line Interfaces.....	5
Lab Exercise – 1.....	5
Answer Lab Exercise – 1	5
Module 5 – Database Clusters.....	8
Lab Exercise – 1.....	8
Answer - Lab Exercise-1.....	8
Module 6 - Configuration.....	10
Lab Exercise - 1	10
Answer Lab Exercise -1.....	10
Lab Exercise - 2	10
Answer Lab Exercise – 2	11
Lab Exercise - 3	12
Lab Exercise - 4	12
Module 7 - Data Dictionary	14
Lab Exercise - 1	14
Answer Lab Exercise - 1.....	14
Lab Exercise - 2	14
Answer Lab Exercise - 2.....	14
Lab Exercise - 3	15
Answer Lab Exercise - 3.....	15
Lab Exercise - 4	17
Answer Lab Exercise - 4.....	17
Module 8 - Creating and Managing Databases.....	18
Lab Exercise - 1	18
Lab Exercise - 2	19
Answer Lab Exercise - 2.....	19
Module 9 - Security	21
Lab Exercise - 1	21



Answer Lab Exercise - 1	21
Lab Exercise - 2	21
Answer Lab Exercise - 2	21
Lab Exercise - 3	22
Answer Lab Exercise - 3	22
Module 10 – Monitoring and Admin Tools.....	23
Lab Exercise - 1	23
Answer Lab Exercise - 1	23
Lab Exercise - 2	28
Answer Lab Exercise - 2	28
Lab Exercise - 3	32
Answer Lab Exercise - 3	32
Lab Exercise - 4	33
Answer Lab Exercise - 4	33
Module 11 - SQL Primer	35
Lab Exercise - 1	35
Answer Lab Exercise - 1	35
Lab Exercise - 2	35
Answer Lab Exercise - 2	36
Lab Exercise - 3	36
Answer Lab Exercise - 3	36
Module 12 - Backup Recovery	37
Lab Exercise - 1	37
Answer Lab Exercise - 1	37
Lab Exercise - 2	38
Answer Lab Exercise - 2	38
Lab Exercise - 3	38
Answer Lab Exercise - 3	39
Lab Exercise - 4	39
Answer Lab Exercise - 4	39
Lab Exercise - 5	41
Answer Lab Exercise - 5	41
Module 13 - Routine Maintenance Tasks.....	43
Lab Exercise – 1	43



Answer Lab Exercise - 1	43
Lab Exercise - 2	44
Answer Lab Exercise - 2	44
Module 14 - Moving Data	45
Lab Exercise - 1	45
Answer Lab Exercise - 1	45



Answer Key for Foundations PostgreSQL Administration Labs – 15

Module 3 - PostgreSQL Installation

Lab Exercise - 1

In this lab exercise you can practice installing PostgreSQL on Linux

1. Choose the platform on which you want to install PostgreSQL
2. Download the PostgreSQL installer from the postgresql.org for the chosen platform
3. Prepare the platform for installation
4. Install PostgreSQL and connect to a database using psql

Answer Lab Exercise 1 (RHEL/Rocky/OL Linux 9)

(SKIP THIS SECTION IF YOU HAD ALREADY INSTALLED POSTGRESQL ON LINUX)

1. Connect to your Linux machine as root or sudo user
2. Use useradd command to create a new user:

```
[root@Base ~]# useradd postgres
[root@Base ~]# passwd postgres
Changing password for user postgres.
New password:
Retype new password:
passwd:
all authentication tokens updated successfully.
```

3. Install the repository RPM:
`sudo dnf install -y https://download.postgresql.org/pub/repos/yum/reporpms/EL-9-x86_64/pgdg-redhat-repo-latest.noarch.rpm`
4. Disable the built-in PostgreSQL module:
`sudo dnf -qy module disable postgresql`
5. Install PostgreSQL:
`sudo dnf install -y postgresql16-server`
6. Create a database cluster and start the cluster using services:
`sudo /usr/pgsql-16/bin/postgresql-16-setup initdb`
7. Enable autostart for PostgreSQL service:
`sudo systemctl enable postgresql-16`
8. Start PostgreSQL Service:
`sudo systemctl start postgresql-16`



Module 4 - User Tools - Command Line Interfaces

Lab Exercise – 1

1. Connect to a database using psql.
2. Switch databases.
3. Describe the `customers` table.
4. Describe the `customers` table including description.
5. List all databases.
6. List all schemas.
7. List all tablespaces.
8. Execute a sql statement, saving the output to a file.
9. Do the same thing, just saving data, not the column headers.
10. Create a script via another method and execute from psql.
11. Turn on the expanded table formatting mode.
12. Lists tables, views and sequences with their associated access privileges.
13. View the current working directory.

Answer Lab Exercise – 1

NOTE: ALL PRACTICE LABS OF MODULE 3 & 4 MUST BE DONE PRIOR TO THIS LAB EXERCISE

1. Connect to database using psql.
Type the following command:
`psql -h localhost -p 5432 postgres postgres` then type the **postgres** password.
2. Switch to the edbstore database.
Type the following command:
`\c edbstore edbuser` then type **edbuser** password.
3. Describe the customers table.
Type
`\d customers`
4. Describe the customers table including description.
Type the following command: `\d+ customers`
5. List all databases.
Type the following command: `\l`
6. List all schemas. Type the following command: `\dn`
7. List all tablespaces. Type the following command: `\db`
8. Execute an SQL statement, to save the output to a file.



Type the following command:

```
\o customer_data.txt
SELECT * FROM customers;
\o
```

9. Do the same thing, just saving data, not the column headers.

Type the following command:

```
\t
\o customer_data.txt
SELECT * FROM customers;
\o
\t
```

10. Exit out of psql then create a script via another method, and execute from psql.

a. Exit .Type `\q`

b. Create the sql file.

Type:

- i. `vi emp.sql`
- ii. `SELECT * FROM emp;`
- iii. `<ESC>:wq <Enter>`

c. Connect back to psql and execute the script.

Type

`psql -f emp.sql -d edbstore -U edbuser` then enter the **edbuser** password

d. Connect to the edbstore database using psql.

Type

`psql -d edbstore -U edbuser`

11. Turn on the expanded table formatting mode.

Type the following command:

```
\x
SELECT * FROM dept;
\x
```

12. Lists tables, views, and sequences with their associated access privileges.

Type the following command: `\dp`

13. View the current working directory. Type `\! pwd`

Exit psql. Type `\q`





Module 5 – Database Clusters

Lab Exercise – 1

1. A new website is to be developed for an online music store
 - Create a new cluster with data directory `/edbdata` and ownership of `postgres` user
 - Start your `edbdata` cluster
 - Reload your cluster with `pg_ctl` utility and using `pg_reload_conf()` function
 - Stop your `edbdata` cluster with fast mode

Note: Answer of this Lab Exercise is given assuming `PATH` variable is set in **.bash_profile** or can be set using `export PATH=/usr/pgsql-16/bin:$PATH`

Answer - Lab Exercise-1

1. Open a terminal window. Type `sudo mkdir /edbdata` then enter the **sudo** user password.
Type command:
`sudo mkdir /edbdata`
2. Change the ownership to `postgres` user.
Type the following command:
`sudo chown postgres:postgres /edbdata`
3. Login as `postgres`.
Type `su - postgres` and enter the **postgres** user password
4. Create and initialize the cluster.
Type the following command:
`initdb -D /edbdata -W` then enter the **database superuser** password twice. The default database superuser is “`postgres`”
5. Change the port.
Type
`vi /edbdata/postgresql.conf`, then click the `<INSERT>` key
6. Change the port to 5433 . Uncomment the parameter and set the port
`port = 5433`
7. Save and close the file.
Type `<ESC>:wq <Enter>`
8. Start the cluster.



Type `pg_ctl -D /edbdata start`

9. Reload the cluster.

Type the following command:

`pg_ctl -D /edbdata reload`

or connect using `psql -p 5433 postgres postgres` and

type the following command:

`SELECT pg_reload_conf();`

10. Stop the cluster.

Type the following command:

`pg_ctl -D /edbdata -mf stop`



Module 6 - Configuration

Lab Exercise - 1

1. You are working as DBA. It is recommended to keep a backup copy of **postgresql.conf** file before making any changes. Make necessary changes in server parameter file for following settings:
 - Allow up to 200 connected users
 - Reserve 10 connection slots for DBA
 - Set the maximum time to complete client authentication to be 10 seconds

Answer Lab Exercise -1

Open a terminal window. Type `su - postgres`

1. Create a copy of config file.

Type the following command:

```
cp /var/lib/pgsql/16/data/postgresql.conf  
~/postgresql.conf.backup
```

Open the **postgresql.conf** file. Type

```
vi /var/lib/pgsql/16/data/postgresql.conf
```

then click the <INSERT> key.

2. Make following Changes:

- a. `max_connections = 200` (remember to uncomment the line)
- b. `superuser_reserved_connections = 10` (remember to uncomment the line)
- c. `authentication_timeout = 10s` (remember to uncomment the line)

3. Save the file and close. Type `<ESC>:wq <Enter>`

4. Restart database cluster to implement the changes.

```
pg_ctl -D /var/lib/pgsql/16/data/ -mf restart
```

Lab Exercise - 2

1. Working as a DBA is a challenging job and to track down certain activities on the database server logging has to be implemented. Go through the server parameters that control logging and implement the following:



- Save all the error message in a file inside the `log` folder in your cluster data directory (e.g. `/edbdbdata`)
- Log all queries which are taking more than 5 seconds to execute, and their time
- Log the users who are connecting to the database cluster
- Make the above changes and verify them

Answer Lab Exercise – 2

1. Open a terminal and login as the postgres user. Type `su - postgres` then the password.
2. Open the **postgresql.conf** file. Type `vi /var/lib/pgsql/16/data/postgresql.conf`
3. Make following changes:
 - a. `log_directory = 'log'`
 - b. `log_min_duration_statement = 5000`
 - c. `log_connections = on`
4. Save the file and close. Type `<Esc>:wq <Enter>`
5. Reload postgres cluster to implement the changes. Type `pg_ctl -D /var/lib/pgsql/16/data reload`
6. Verify the changes by connecting to a database in your database cluster. Type `psql -p 5432 postgres postgres`
 - a. Type the following command:
`SHOW log_connections;`
`SHOW log_min_duration_statement;`
`SHOW log_directory;`

```
postgres=# show log_connections;
log_connections
-----
on
(1 row)

postgres=# show log_min_duration_statement;
log_min_duration_statement
-----
5s
(1 row)

postgres=# show log_directory;
log_directory
-----
log
(1 row)
```

Exit psql type `\q`



Lab Exercise - 3

1. Perform the following changes recommended by a senior DBA and verify them.

Set:

- Shared buffer to 256MB
- Effective cache for indexes to 512MB
- Maintenance memory to 64MB
- Temporary memory to 8MB

Answer Lab Exercise - 3

Open a terminal and logon as the postgres user. Type
`su - postgres` then the password.

Open the postgresql conf file. Type
`vi /var/lib/pgsql/16/data/postgresql.conf`

then type `<i>` to go into insert mode

1. Make following Changes:
 - a. `shared_buffers = 256MB`
 - b. `maintenance_work_mem = 64MB`
 - c. `effective_cache_size = 512MB`
 - d. `work_mem = 8MB`

Save the file and close. Type

`<ESC>:wq <Enter>`

2. Restart postgres cluster to implement the changes.
Type the following command:
`pg_ctl -D /var/lib/pgsql/16/data -mf restart`

Lab Exercise - 4

1. Vacuuming is an important maintenance activity and needs to be properly configured. Change the following **autovacuum** parameters on the production server. Set:
 - Autovacuum workers to 6
 - Autovacuum threshold to 100
 - Autovacuum scale factor to 0.3
 - Auto analyze threshold to 100
 - Autovacuum cost limit to 100



Answer Lab Exercise - 4

Open a terminal and logon as the postgres user.

Type

`su - postgres` then the password.

Open the postgresql conf file. Type `<i>` to go into insert mode

`vi /var/lib/pgsql/16/data/postgresql.conf`

1. Make following Changes:

- a. `autovacuum_max_workers = 6`
- b. `autovacuum_vacuum_threshold = 100`
- c. `autovacuum_vacuum_scale_factor = 0.3`
- d. `autovacuum_analyze_threshold = 100`
- e. `autovacuum_vacuum_cost_limit = 100`

2. Save the file and close. Type `<ESC>:wq <Enter>`

3. Reload postgres cluster to implement the changes. Type

`pg_ctl -D /var/lib/pgsql/16/data/ reload`



Module 7 - Data Dictionary

Lab Exercise - 1

1. You are working with different schemas in a database. You need to determine all the schemas in your search path. Write a query to find the list of schemas currently in your search path.

Answer Lab Exercise - 1

Open a terminal and login as the postgres user. Type
`su - postgres` then the password.

Go to psql in the edbstore database. Type
`psql edbstore edbuser` and type the **edbuser** password

1. Review the current search path. Type the following command:
`SELECT current_schemas(true);`

```
edbstore=> select current_schemas(true);
           current_schemas
-----
{pg_catalog,edbuser,public}
(1 row)
```

2. Write a query to find the list of schemas currently in your search path. Type
`SHOW search_path;`

```
edbstore=> show search_path;
           search_path
-----
"$user", public
(1 row)
```

Exit psql type `\q`

Lab Exercise - 2

1. You need to determine the names and definitions of all the views in your schema. Create a report that retrieves view information - the view name and definition text.

Answer Lab Exercise - 2

Open a terminal and login as the postgres user. Type
`su - postgres` then the password.

Go to psql in the edbstore database. Type
`psql edbstore edbuser` and type the **edbuser** password



1. Create a report that retrieves view information: the view name and definition text.

Type the following command:

```
SELECT * FROM pg_views WHERE schemaname='edbuser';
```

edbstore=> SELECT * FROM pg_views where schemaname='edbuser';			
schemaname	viewname	viewowner	definition
edbuser	salesemp	edbuser	SELECT emp.empno, emp.ename, emp.hiredate, emp.sal, emp.comm FROM emp WHERE ((emp.job)::text = 'SALESMAN'::text);
(1 row)			

Exit psql type \q

Lab Exercise - 3

1. Create report of all the users who are currently connected. The report must display total session time of all connected users.
2. You found that a user has connected to the server for a very long time and have decided to gracefully kill its connection. Write a statement to perform this task.

Answer Lab Exercise - 3

Open 2 terminal windows

Login as the postgres user in first.

Type `su - postgres` then the password.

Login as the postgres user.

Type `su - postgres` then the password.

1. In first terminal, go to psql in the edbstore database as the edbuy user
Type the following command:
`psql edbstore edbuy` and type the **edbuy** password `lion`
2. In second terminal, go to psql in the edbstore database.
Type the following command:
`psql edbstore edbuser` and type the **edbuser** password
3. Open a third terminal and logon as the postgres user.
Type `su - postgres` then the password.



4. In third terminal, go to psql in the postgres database. Type `psql postgres postgres` and type the **postgres** password
5. Create report of all the users who are currently connected. The report must display total session time of all connected users.

Type the following command:

```
SELECT username,now()-backend_start AS "Total Connection Time", pid FROM pg_stat_activity;
```

```
postgres=# select username,now()-backend_start AS "Total Connection Time", pid FROM pg_stat_activity;
username | Total Connection Time | pid
-----|-----|-----
postgres | 00:12:32.014248       | 8686
postgres | 00:12:32.010946       | 8688
edbuser  | 00:01:14.473833       | 8973
postgres | 00:01:02.840074       | 8985
postgres | 00:12:32.017165       | 8684
postgres | 00:12:32.012215       | 8683
postgres | 00:12:32.015841       | 8685
(7 rows)
```

6. Run the user connection report again.

Type the following command:

```
SELECT username,now()-backend_start AS "Total Connection Time", pid FROM pg_stat_activity;
```

```
postgres=# select username,now()-backend_start AS "Total Connection Time", pid FROM pg_stat_activity;
username | Total Connection Time | pid
-----|-----|-----
postgres | 00:13:43.656694       | 8686
postgres | 00:13:43.653392       | 8688
edbuser  | 00:02:26.116279       | 8973
postgres | 00:02:14.48252        | 8985
postgres | 00:13:43.659611       | 8684
postgres | 00:13:43.654661       | 8683
postgres | 00:13:43.658287       | 8685
(7 rows)
```

7. You found the the **edbuser** user has been connected to the server for too long. Gracefully kill edbuser's connection. Write a statement to perform this task.

Type the following command:

```
SELECT pg_terminate_backend(edbuser pid);
```

```
postgres=# select pg_terminate_backend(8973);
pg_terminate_backend
-----
t
(1 row)
```



8. Run the user connection report again.

Type the following command:

```
SELECT username,now()-backend_start AS "Total Connection Time", pid FROM pg_stat_activity;
```

```
postgres=# select username,now()-backend_start AS "Total Connection Time", pid FROM pg_stat_activity;
username | Total Connection Time | pid
-----+-----+-----
postgres | 00:15:48.864074      | 8686
postgres | 00:15:48.860772      | 8688
postgres | 00:04:19.6899        | 8985
          | 00:15:48.866991      | 8684
          | 00:15:48.862041      | 8683
          | 00:15:48.865667      | 8685
(6 rows)
```

Exit psql type \q.

Lab Exercise - 4

1. Write a query to display the name and size of all the databases in your cluster. Size must be displayed using a meaningful unit.

Answer Lab Exercise - 4

Open a terminal and login as the postgres user. Type
su - postgres then the password.

Go to psql in the postgres database. Type
psql postgres postgres and type the **postgres** password

1. Write a query to display name and size of all the databases in your cluster. Size must be displayed using a meaningful unit.

Type the following command:

```
SELECT datname,
pg_size_pretty(pg_database_size(datname)) AS size from
pg_database;
```

```
postgres=# SELECT datname,pg_size_pretty(pg_database_size(datname)) AS size from pg_database;
datname | size
-----+-----
postgres | 7939 kB
edbstore | 25 MB
template1 | 7801 kB
template0 | 7801 kB
(4 rows)
```

Exit psql type \q



Module 8 - Creating and Managing Databases

Lab Exercise - 1

1. An e-music online store website application developer wants to add an online buy/sell facility and has asked you to separate all tables used in online transactions. Here you have suggested to use schemas. Implement the following suggested options:
 - Create an `ebuy` user with password `'lion'`.
 - Create an `ebuy` schema which can be used by user `ebuy`.
 - Login as the `ebuy` user, create a table `sample1` and check whether that table belongs to the `ebuy` schema or not.

Answer Lab Exercise - 1

Connect postgres database using psql utility:

```
psql postgres postgres
```

```
[postgres@pgsrv1 ~]$ psql postgres postgres
psql (16.0)
Type "help" for help.

postgres=#
```

1. Create a user `ebuy` with the password `lion`.
Type the following command:
`CREATE USER ebuy Password 'lion';`
2. Create the schema `ebuy`.
Type the following command:
`CREATE SCHEMA ebuy authorization ebuy;`
3. Switch to the `ebuy` user.
Type the following command:
`\c postgres ebuy` and the `lion` password.

Create a table `sample1`.

Type the following command:

```
CREATE table sample1(id numeric, name varchar);
```

Review the tables in the schema. Type `\dt`



```
postgres=# CREATE USER ebuy Password 'lion';
CREATE ROLE
postgres=# CREATE SCHEMA ebuy authorization ebuy;
CREATE SCHEMA
postgres=# \c postgres ebuy
Password for user ebuy:
You are now connected to database "postgres" as user "ebuy".
postgres=> CREATE table sample1(id numeric, name varchar);
CREATE TABLE
postgres=> \dt
               List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 ebuy   | sample1 | table | ebuy
 public | Test    | table | postgres
(2 rows)
```

Quit out of psql type \q

Lab Exercise - 2

1. Retrieve a list of databases using a SQL query.
2. Retrieve a list of databases using the psql meta command.
3. Retrieve a list of tables in the `edbstore` database and check which schema and owner they have.

Answer Lab Exercise - 2

Connect to `edbstore` database.

Type the following command:

`psql -d edbstore -U edbuser` then enter the **edbuser** password.

1. Retrieve a list of databases. Type
`SELECT datname FROM pg_database;`

```
edbstore=> select datname from pg_database;
 datname
-----
 postgres
 template1
 template0
 edbstore
 pgtest
(5 rows)
```



2. List the databases using a psql meta command. Type \l.

```
edbstore=> \l
```

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU Locale	ICU Rules	Access privileges	
edbstore	edbuser	UTF8	libc	C.UTF-8	C.UTF-8				
pgtest	edbuser	UTF8	libc	C.UTF-8	C.UTF-8				
postgres	postgres	UTF8	libc	C.UTF-8	C.UTF-8				
template0	postgres	UTF8	libc	C.UTF-8	C.UTF-8				=c/postgres +
template1	postgres	UTF8	libc	C.UTF-8	C.UTF-8				postgres=CTc/postgres +
									=c/postgres +
									postgres=CTc/postgres

(5 rows)

3. Retrieve a list of tables in edbstore database and check which schema and owner they have. Type \dt

```
edbstore=> \dt
```

List of relations			
Schema	Name	Type	Owner
edbuser	categories	table	edbuser
edbuser	cust_hist	table	edbuser
edbuser	customers	table	edbuser
edbuser	dept	table	edbuser
edbuser	emp	table	edbuser
edbuser	inventory	table	edbuser
edbuser	job_grd	table	edbuser
edbuser	jobhist	table	edbuser
edbuser	locations	table	edbuser
edbuser	orderlines	table	edbuser
edbuser	orders	table	edbuser
edbuser	products	table	edbuser
edbuser	reorder	table	edbuser

(13 rows)

Exit out of psql. Type \q



Module 9 - Security

Lab Exercise - 1

1. You are working as a Postgres DBA. Your server box has 2 network cards with IP addresses 192.168.30.10 and 10.4.2.10. 192.168.30.10 is used for the internal LAN and 10.4.2.10 is used by the web server to connect users from an external network. Your server should accept TCP/IP connections both from internal and external users.
 - Configure your server to accept connections from external and internal networks.

Answer Lab Exercise - 1

Open a terminal and login as the postgres user.

Type

`su - postgres` then the password.

1. Open the **postgresql.conf** file.

Type the following command:

```
vi /var/lib/pgsql/16/data/postgresql.conf
```

2. Make following Changes Type `<i>` to get in insert mode

```
listen_addresses='*'
```

3. Save the file and close Type

```
<ESC>:wq <Enter>
```

4. Restart postgres cluster to implement the changes.

Type the following command:

```
pg_ctl -D /var/lib/pgsql/16/data/ -mf restart
```

Lab Exercise - 2

1. A new developer has joined the team, and his ID number is 89.
 - Create a new user by name `dev89` and password `password89`.
 - Then assign the necessary privileges to `dev89` so that he can connect to the `edbstore` database and view all tables.

Answer Lab Exercise - 2

Open a terminal and login as the postgres user.

Type

`su - postgres` then the password.

Go to psql in the postgres database. Type

`psql postgres postgres` and type the **postgres** password.



1. Create the user dev89.

Type the following command:

```
CREATE USER dev89 PASSWORD 'password89';
```

2. Grant connection rights to the dev89 user.

Type the following command:

```
GRANT CONNECT ON DATABASE edbstore to dev89;
```

Switch the edbstore database as the postgres user. Type

```
\c edbstore postgres
```

3. Grant usage rights to dev89 user on the edbuser schema.

Type the following command:

```
GRANT USAGE ON SCHEMA edbuser to dev89;
```

4. Grant select, insert, delete and update on all the edbuser tables.

Type the following command:

```
GRANT SELECT, INSERT, DELETE, UPDATE ON ALL TABLES IN  
SCHEMA edbuser to dev89;
```

5. Switch to the edbstore database as dev89 user.

Type the following command:

```
\c edbstore dev89 then enter the password for dev89.
```

6. Check to see dev89 can access the tables.

Type the following command:

```
SELECT * FROM edbuser.dept;
```

Exit out of psql

Type \q

Lab Exercise - 3

1. A new developer joins e-music corp. They have an IP address 192.168.30.89. They are not able to connect from his machine to the Postgres server and gets the following error on the server:

```
FATAL: no pg_hba.conf entry for host "192.168.30.89", user  
"dev89", database "edbstore", SSL off
```

2. Configure your server so that the new developer can connect from their machine.

Answer Lab Exercise - 3

Open a terminal and login as the postgres user. Type

```
su - postgres then the password.
```



1. Edit the **pg_hba.conf** file to fix the error.
Type the following command:

```
vi /var/lib/pgsql/16/data/pg_hba.conf
```
2. Add following entry to the end of the **pg_hba.conf** file.
Type **<i>** to get into insert mode

```
host edbstore dev89 192.168.30.89/32 md5
```


Save the file and close the file.
Type

```
<ESC>:wq <Enter>
```
3. Reload postgres cluster to implement the changes.
Type the following command:

```
pg_ctl -D /var/lib/pgsql/16/data/ reload
```

Module 10 – Monitoring and Admin Tools

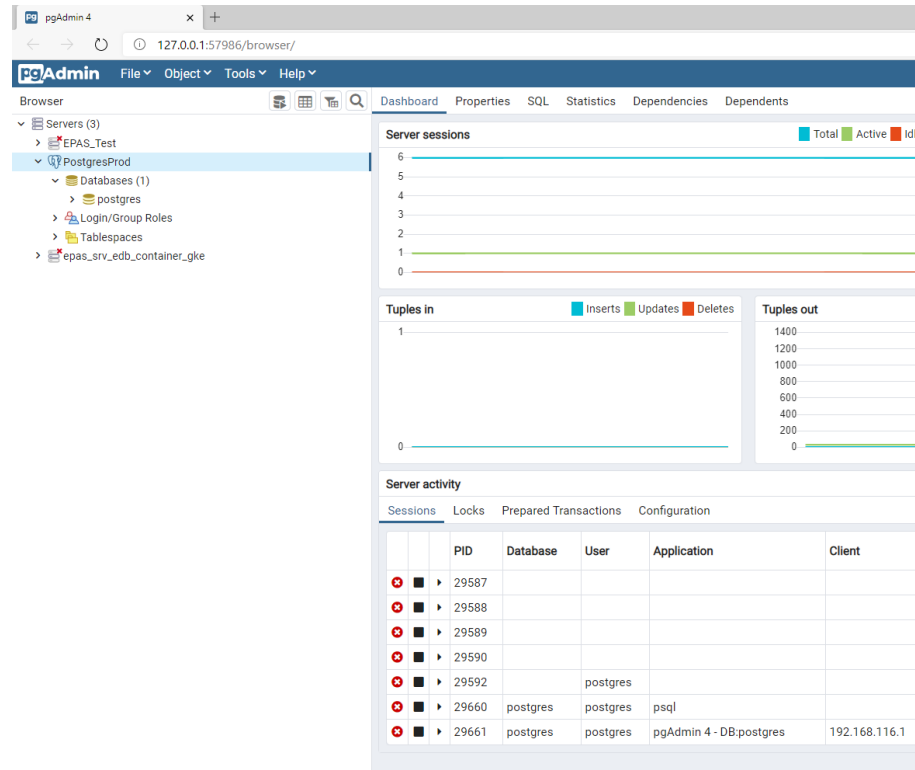
Lab Exercise - 1

1. Open pgAdmin 4 and connect to the default PostgreSQL database cluster
 - Create a user named `pguser`
 - Create a database named `pgdb` owned by `pguser`
 - After creating the `pgdb` database change its connection limit to 4
 - Create a schema named `pguser` inside the `pgdb` database
 - The schema owner should be `pguser`

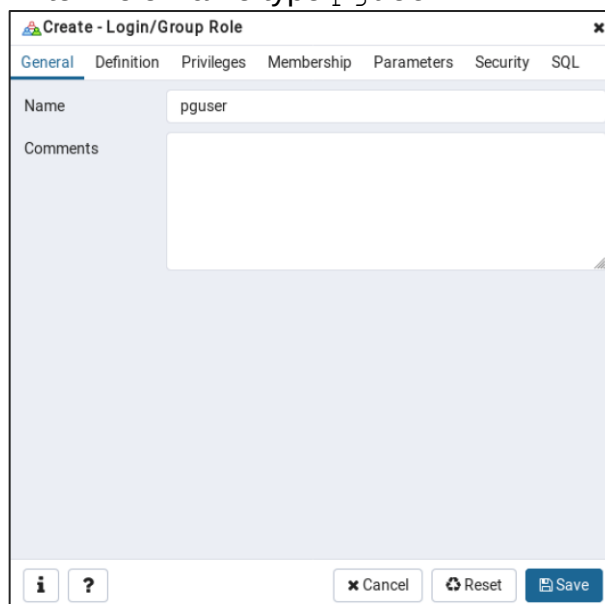
Answer Lab Exercise - 1

Lab Exercise - 1

1. PgAdmin 4 on Linux can be installed using instruction available on Practice Lab for this module.
2. Open PgAdmin 4 and connect to the default Postgres database cluster

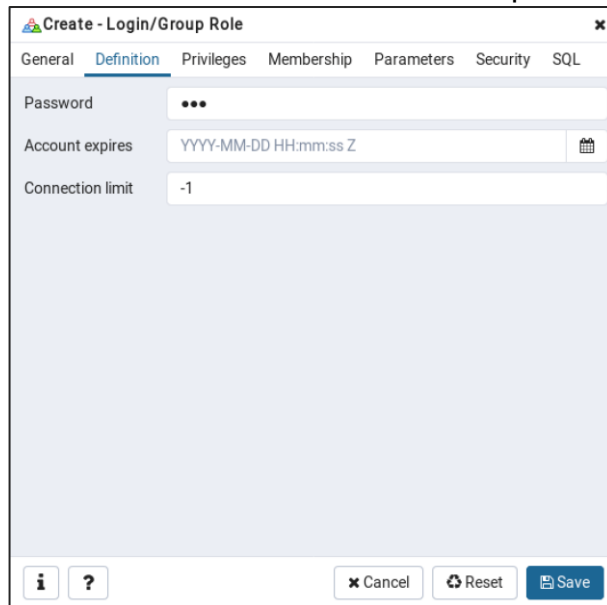


3. Create user pguser
 - a. Right click on **Login/Group Roles** and choose **Create:Login/Group Roles**.
 - b. Enter Role Name type `pguser`.



The screenshot shows the 'Create - Login/Group Role' dialog box in pgAdmin 4. The 'General' tab is selected. The 'Name' field contains 'pguser'. The 'Comments' field is empty. At the bottom, there are buttons for 'Cancel', 'Reset', and 'Save'.

- c. Click the **Definition** tab and enter a password for `pguser`.



Create - Login/Group Role

General **Definition** Privileges Membership Parameters Security SQL

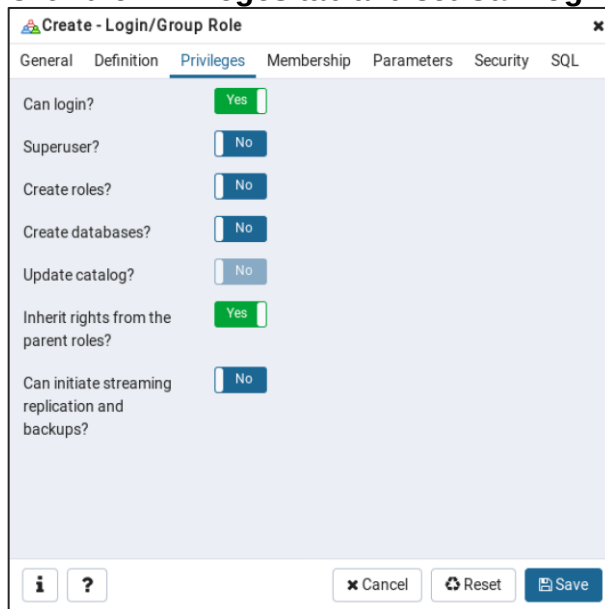
Password:

Account expires: 

Connection limit:

   Cancel  Reset  Save

- d. Click the **Privileges** tab and set **Can login** to **Yes** then click **Save**.



Create - Login/Group Role

General Definition **Privileges** Membership Parameters Security SQL

Can login? ☒ Yes

Superuser? ☐ No

Create roles? ☐ No

Create databases? ☐ No

Update catalog? ☐ No

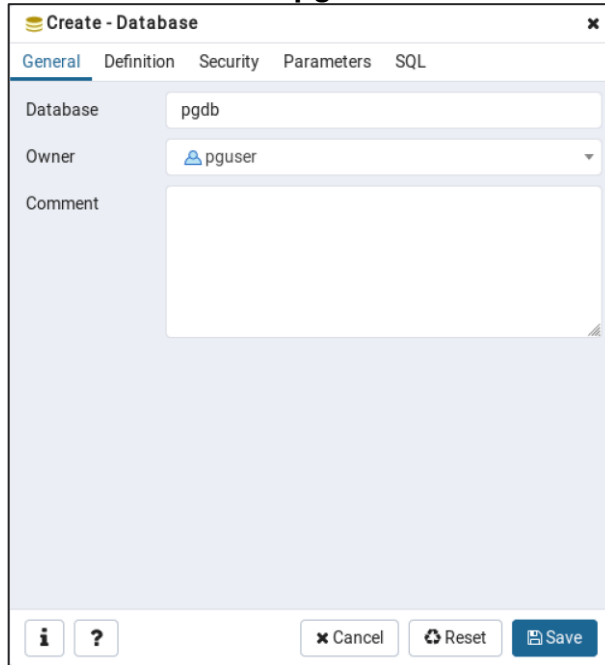
Inherit rights from the parent roles? ☒ Yes

Can initiate streaming replication and backups? ☐ No

   Cancel  Reset  Save

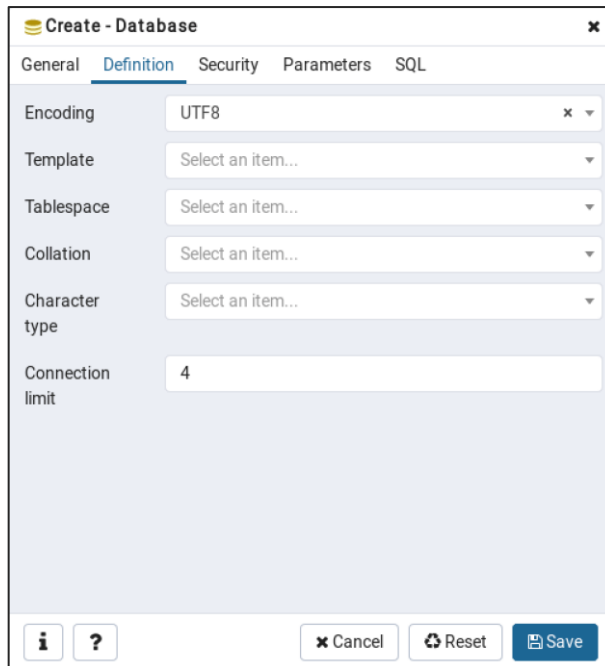
4. Create a database called `pgdb` owned by `pguser`.
 - a. Right click **Databases** and choose **Create: Database**.
 - b. Enter the database name type `pgdb`.

- c. Set the owner to be **pguser** and click **Save**.



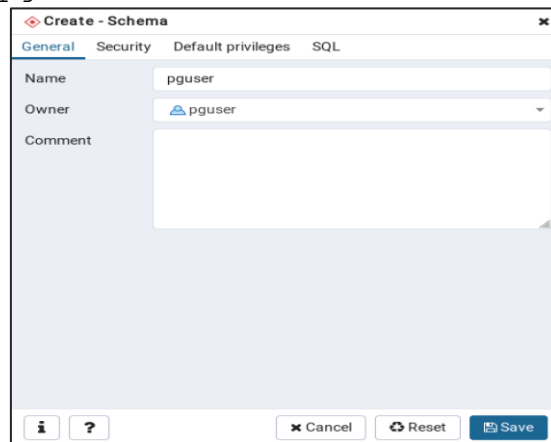
The 'Create - Database' dialog box is shown with the 'General' tab selected. The 'Database' field contains 'pgdb'. The 'Owner' dropdown menu is set to 'pguser'. The 'Comment' field is empty. At the bottom, there are buttons for 'Cancel', 'Reset', and 'Save'.

5. Change the database connection limit to 4.
- a. Right Click **pgdb** database and click **Properties**.
- b. Click **Definition** tab change the **Connection Limit** to 4 then click **Save**.



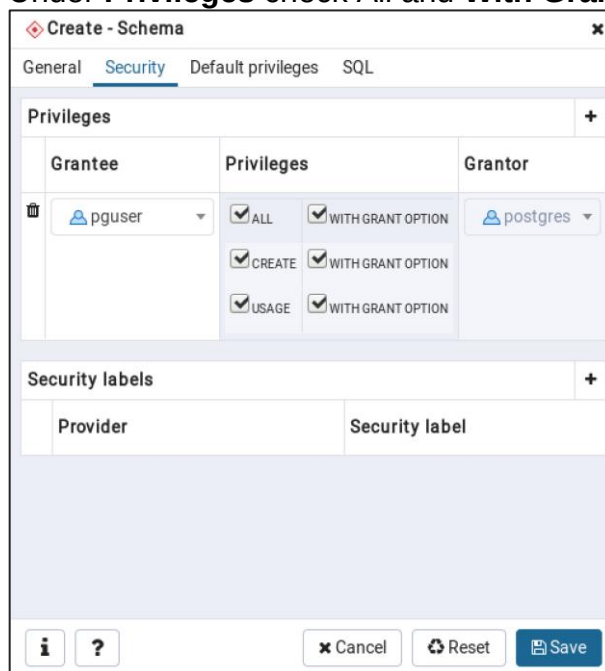
The 'Create - Database' dialog box is shown with the 'Definition' tab selected. The 'Encoding' is set to 'UTF8'. The 'Template', 'Tablespace', 'Collation', and 'Character type' fields are all set to 'Select an item...'. The 'Connection limit' field is set to '4'. At the bottom, there are buttons for 'Cancel', 'Reset', and 'Save'.

6. Create a schema in the pgdb database called pguser. Set the owner to be pguser.
 - a. Right Click **Schemas** under pgdb database and click **Create: Schema**
 - b. Enter the **Schema Name** as pguser and enter the **Owner** as pguser,



The 'Create - Schema' dialog box is shown with the 'General' tab selected. The 'Name' field contains 'pguser'. The 'Owner' dropdown menu is set to 'pguser'. The 'Comment' field is empty. At the bottom, there are buttons for 'Cancel', 'Reset', and 'Save'.

- c. Click the **Security** tab then under **Grantee** choose pguser. Under **Privileges** check All and **With Grant Option**. Then click **Save**.



The 'Create - Schema' dialog box is shown with the 'Security' tab selected. The 'Privileges' section is expanded, showing a table with columns: Grantee, Privileges, and Grantor. The 'Grantee' dropdown is set to 'pguser'. The 'Privileges' section has checkboxes for 'ALL', 'CREATE', and 'USAGE', each with a 'WITH GRANT OPTION' checkbox. The 'Grantor' dropdown is set to 'postgres'. Below the privileges table is a 'Security labels' section with columns for 'Provider' and 'Security label'. At the bottom, there are buttons for 'Cancel', 'Reset', and 'Save'.

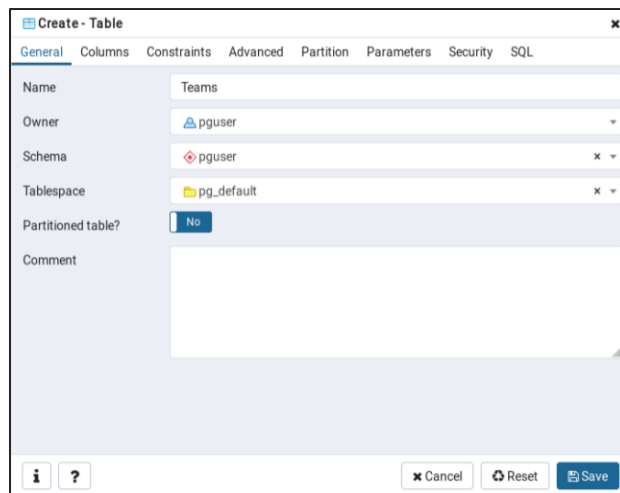
Grantee	Privileges	Grantor
pguser	☒ ALL ☒ WITH GRANT OPTION ☒ CREATE ☒ WITH GRANT OPTION ☒ USAGE ☒ WITH GRANT OPTION	postgres

Lab Exercise - 2

1. You have created the `pgdb` database with the `pguser` schema. Create following objects in the `pguser` schema:
 - Table - Teams with columns `TeamID`, `TeamName`, `TeamRatings`
 - Sequence - `seq_teamid` start value - 1 increment by 1
 - Columns - Change the default value for the `TeamID` column to `seq_teamid`
 - Constraint - `TeamRatings` must be between 1 and 10
 - Index - Primary Key `TeamID`
 - View - Display all teams in ascending order of their ratings. Name the view as `vw_top_teams`

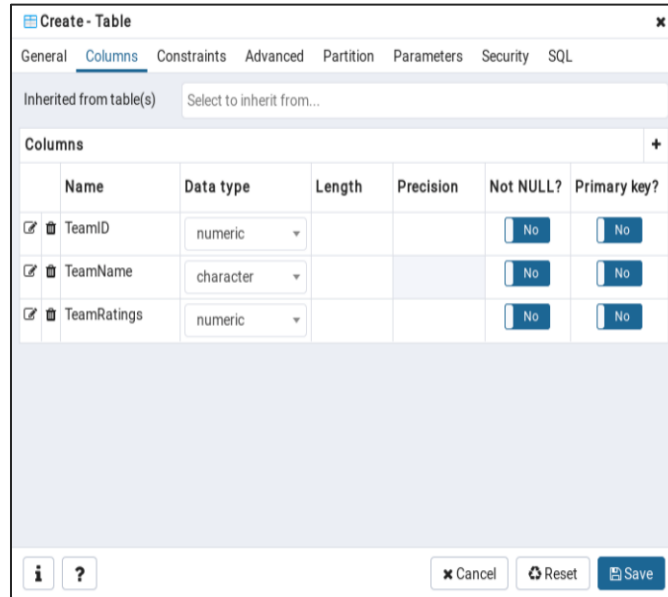
Answer Lab Exercise - 2

1. Open **PgAdmin 4**. Expand Databases. Go to **pgdb** then to **Schemas** and to **pguser**, created in the last exercise.
2. Create the Teams Table.
 - a. Right click **Tables** and click **Create: Table**.
 - b. Enter **Table name** Teams and set the **Owner** to be `pguser`.



- c. Click **Columns** tab and enter the following column names and data types then click **Save**. For each new column click the add button  in the right corner.
 - i. `TeamID`, numeric
 - ii. `TeamName`, character varying

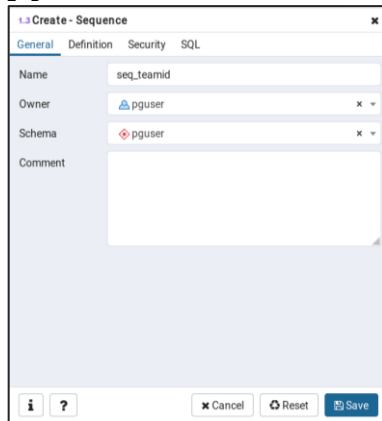
iii. TeamRatings, numeric



The 'Create - Table' dialog box is shown with the 'Columns' tab selected. It features a table with three columns: Name, Data type, and Length. The 'TeamID' column is set to 'numeric', 'TeamName' to 'character', and 'TeamRatings' to 'numeric'. All columns are set to 'Not NULL?' and 'Primary key?'.

Name	Data type	Length	Precision	Not NULL?	Primary key?
TeamID	numeric			No	No
TeamName	character			No	No
TeamRatings	numeric			No	No

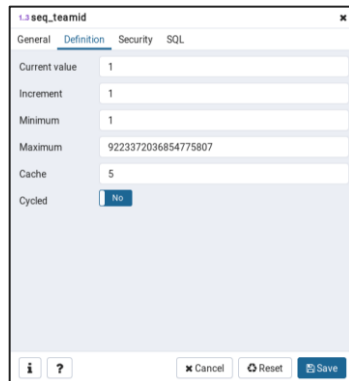
3. Create a Sequence **seq_teamid** with a start value of 1 and increment by 1
 - a. Right click in **Sequences** and click **Create: Sequence**.
 - b. Enter **Sequence Name** `seq_teamid` and set the **Owner** to be `pguser`.



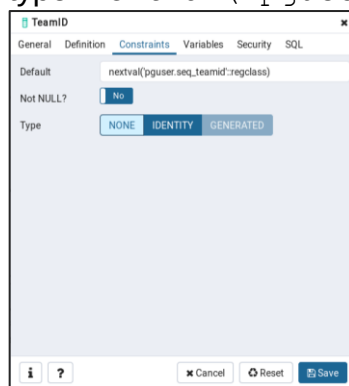
The 'Create - Sequence' dialog box is shown with the 'General' tab selected. It contains fields for Name, Owner, Schema, and Comment. The Name is set to 'seq_teamid', Owner to 'pguser', and Schema to 'pguser'.

Name	Owner	Schema	Comment
seq_teamid	pguser	pguser	

- c. Click on **Definition** tab and enter the **Increment** to be 1 and the **Start** to be 1 then click **Save**.

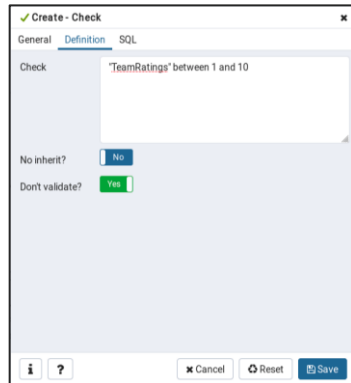


4. Change default value for TeamID column to be seq_teamid.
- Expand the **Teams** table and click onto columns. Right click the **teamid** column and choose **Properties**.
 - Click **Constraints** tab and set the Default value to be seq_teamid type `nextval('pguser.seq_teamid')` then click **Save**.



5. Set the constraint for TeamRatings to be between 1 and 10.
- Right click on **Constraints** in the `teams` table.
 - Go to **Create: Check**
 - Click on the **Definitions** tab and type "`TeamRatings`" between 1 and 10 then click **Save**.

NOTE: You may skip the name as it will be assigned by DB Server.



✓ Create - Check

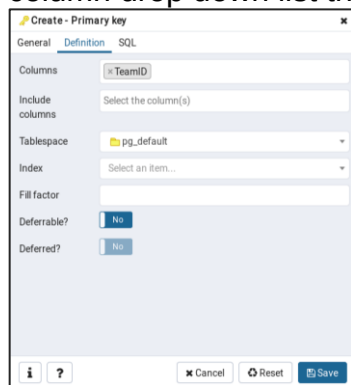
General Definition SQL

Check: "TeamRatings" between 1 and 10

No inherit?

Don't validate?

6. Set the Index Primary Key to be TeamID.
 - a. Right click **Constraints** and choose **Create: Primary Key**.
 - b. Go to the **Definitions** tab and choose the column **TeamID** from column drop down list then click **Save**.



✓ Create - Primary key

General Definition SQL

Columns: TeamID

Include columns: Select the column(s)

Tablespace: pg_default

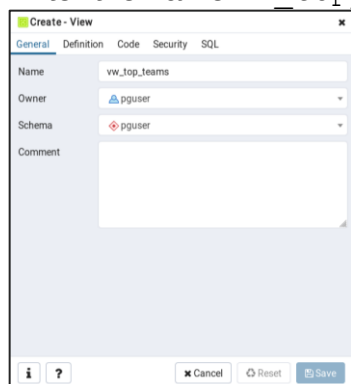
Index: Select an item...

Fill factor:

Deferrable?

Deferred?

7. Create a view that will display all teams in ascending order of their ratings. Name the view as vw_top_teams
 - a. Right click on **Views** in the pguser Schema and click **Create: View**
 - b. Enter the name vw_top_teams and set **Owner** as pguser.



✓ Create - View

General Definition Code Security SQL

Name: vw_top_teams

Owner: pguser

Schema: pguser

Comment:

- c. Click the Code tab and Type
`SELECT "TeamName", "TeamRatings" FROM
pguser."Teams" ORDER BY "TeamRatings" asc` and then
click **Save**.



Lab Exercise - 3

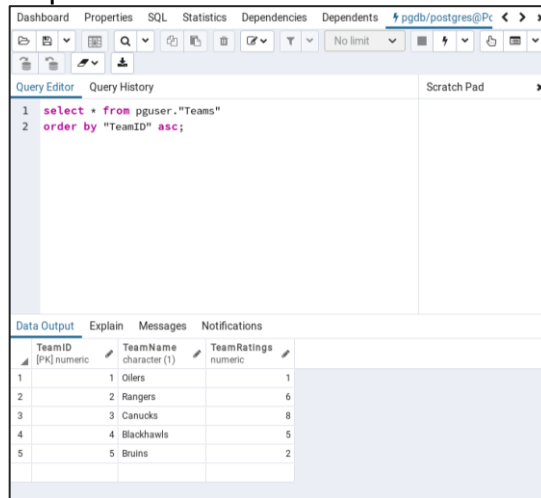
1. View all rows in the `Teams` table.
2. Using the Edit data window you just opened in the previous step, insert the following rows into the `Teams` table:

TeamID	TeamName	TeamRatings
Auto generated	Oilers	1
Auto generated	Rangers	6
Auto generated	Canucks	8
Auto generated	Blackhawks	5
Auto generated	Bruins	2

Answer Lab Exercise - 3

1. View all the rows in the `teams` table. **Right click** the `Teams` table. Choose **View Data** then choose **View All Rows**.
2. In the **Data Output** window from previous step insert the above rows to the `Teams` table.
 - a. Double click each row `TeamName` and enter the value, then double click `TeamRatings` and enter the value.
 - b. Click the Save button.

c. Repeat until all 5 rows are loaded.



The screenshot shows the EDB Query Editor interface. The top bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, and Dependents. The main area is divided into a Query Editor and a Scratch Pad. The Query Editor contains the following SQL query:

```
1 select * from pguser."Teams"
2 order by "TeamID" asc;
```

Below the query editor, the Data Output tab is active, displaying a table with the following data:

TeamID [PK] numeric	TeamName character (1)	TeamRatings numeric
1	Oilers	1
2	Rangers	6
3	Canucks	8
4	Blackhawks	5
5	Bruins	2

3. Click **Save** and close the window.

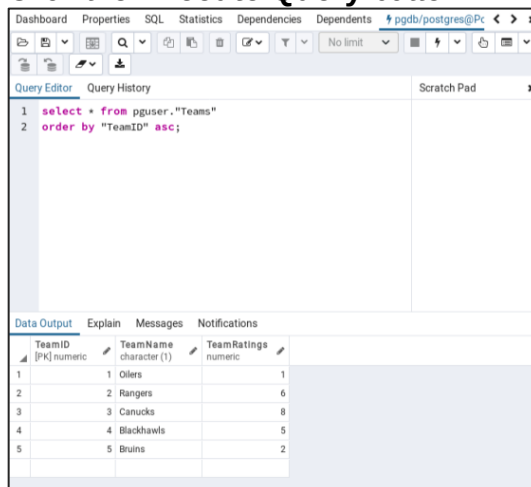
Lab Exercise - 4

1. Connect to the `pgdb` database using the query tool.
2. Using the graphical query builder retrieve all the rows present in the `Teams` table.
3. Using the graphical query builder retrieve all the rows present in view `vw_top_teams`.

Answer Lab Exercise - 4

1. Go to **Tools** and choose **Query Tool**.
2. In the **SQL Editor** Type

```
SELECT * FROM "Teams";
```
3. Click the **Execute Query** button.



The screenshot shows the EDB Query Editor interface. The top bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, and Dependents. The main area is divided into a Query Editor and a Scratch Pad. The Query Editor contains the following SQL query:

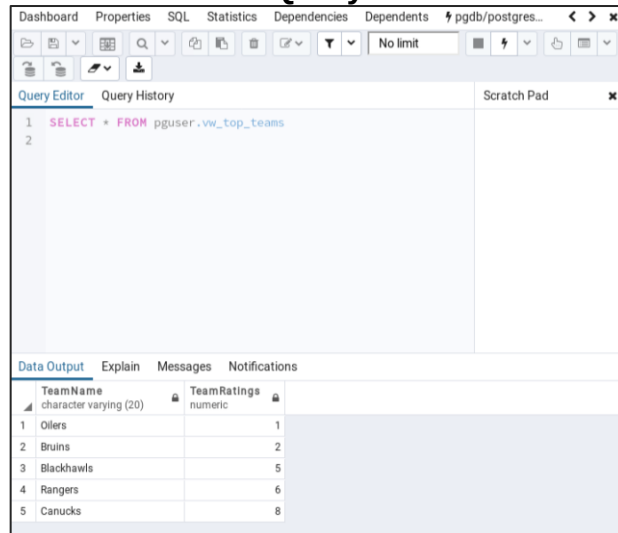
```
1 select * from pguser."Teams"
2 order by "TeamID" asc;
```

Below the query editor, the Data Output tab is active, displaying a table with the following data:

TeamID [PK] numeric	TeamName character (1)	TeamRatings numeric
1	Oilers	1
2	Rangers	6
3	Canucks	8
4	Blackhawks	5
5	Bruins	2

4. In the **SQL Editor** Type
`SELECT * FROM vw_top_teams;`

5. Click the **Execute Query** button.



6. Go to the **File** Menu and choose **Exit**. Click No to save changes message.



Module 11 - SQL Primer

Lab Exercise - 1

Test your knowledge:

1. Initiate a psql session
2. psql commands access the database: True/False
3. The following SELECT statement executes successfully: True/False
`SELECT ename, job, sal AS Salary FROM emp;`
4. The following SELECT statement executes successfully: True/False
`SELECT * FROM emp;`
5. There are coding errors in the following statement. Can you identify them?
`SELECT empno, ename, sal * 12 ANNUAL SALARY FROM emp;`

Answer Lab Exercise - 1

1. Initiate a psql session
 - a. Open a terminal and login as the postgres user. Type `su - postgres` then the password.
 - b. Go to psql in the **edbstore** database. Type `psql edbstore edbuser` and type the **edbuser** password.
2. True
3. True if you are connected as **edbuser** user
4. True if you are connected as **edbuser** user
5. Case sensitive Colum Headings and Column Headings with space must be enclosed in double quotes "
`SELECT empno, ename, sal * 12 "ANNUAL SALARY" FROM edbuser.emp;`

Lab Exercise - 2

1. The staff in the HR department wants to hide some of the data in the `EMP` table. They want a view called `EMPVU` based on the employee numbers, employee names, and department numbers from the `EMP` table. They want the heading for the employee name to be `EMPLOYEE`.
2. Confirm that the view works. Display the contents of the `EMPVU` view.
3. Using your `EMPVU` view, write a query for the `SALES` department to display all employee names and department numbers.



Answer Lab Exercise - 2

Open a terminal and login as the postgres user. Type
`su - postgres` then the password.

Go to psql in the edbstore database.

Type the following command:

`psql edbstore edbuser` and type the **edbuser** password

1. Execute following statements:

```
CREATE VIEW empvu as SELECT empno, ename AS employee,  
deptno FROM emp;
```

```
\dv
```

List of relations			
Schema	Name	Type	Owner
edbuser	empvu	view	edbuser
edbuser	salesemp	view	edbuser
(2 rows)			

```
SELECT * FROM empvu;
```

```
SELECT employee, deptno FROM empvu;
```

Lab Exercise - 3

1. You need a sequence that can be used with the primary key column of the dept table. The sequence should start at 60 and have a maximum value of 200. Have your sequence increment by 10. Name the sequence dept_id_seq.
2. To test your sequence, write a script to insert two rows in the dept table.

Answer Lab Exercise - 3

Open a terminal and login as the postgres user.

Type

`su - postgres` then the password.

Go to psql in the edbstore database.

Type `psql edbstore edbuser` and type the **edbuser** password

1. Create the sequence dept_id_seq.



Type the following command:

```
CREATE SEQUENCE dept_id_seq START WITH 60 INCREMENT BY  
10 MAXVALUE 200;
```

2. Write a script to insert two rows in the `dept` table.

Type the following command:

```
INSERT INTO dept VALUES (nextval('dept_id_seq'),  
'SERVICES');  
INSERT INTO dept VALUES  
(nextval('dept_id_seq'), 'CONTROLS');
```

Module 12 - Backup Recovery

Lab Exercise - 1

1. The `edbstore` website database is all setup and as a DBA you need to plan a proper backup strategy and implement it.
 - As the root user, create a folder `/pgbackup` and assign ownership to the Postgres user using the **chown** utility or the Windows security tab in folder properties.
 - Take a full database dump of the `edbstore` database with the **pg_dump** utility. The dump should be in plain text format.
 - Name the dump file as **edbstore_full.sql** and store it in the `/pgbackup` directory.

Answer Lab Exercise - 1

1. Open a terminal and login as the root user. Type `su - root` then type the root password. You can also connect as a sudoer user and execute command using `sudo`
2. Create a folder `/pgbackup` and assign ownership to Postgres user using **chown** utility or windows security tab in folder properties.
Type the following commands.
 - a. `mkdir /pgbackup`
 - b. `chown -R postgres:postgres /pgbackup`
 - c. `su - postgres`
3. Take a full database dump of the `edbstore` database with the `pg_dump` utility. The dump should be in plain text format. Name the dump file as `edbstore_full.sql` and store it in the `/pgbackup` directory.



Type the following command:

```
pg_dump -f /pgbackup/edbstore_full.sql -U postgres  
edbstore Then enter the postgres password.
```

Lab Exercise - 2

1. Take a dump of the `edbuser` schema from the `edbstore` database and name the file as **edbuser_schema.sql**
2. Take a data-only dump of the `edbstore` database, disable all triggers for faster restore, use the `INSERT` command instead of `COPY`, and name the file as **edbstore_data.sql**
3. Take a full dump of `customers` table and name the file as **edbstore_customers.sql**

Answer Lab Exercise - 2

1. Open a terminal and login as the `postgres` user.
Type
`su - postgres` then the password.
2. Take a dump of the `edbuser` schema from the `edbstore` database and name the dump file **edbuser_schema.sql**.
Type the following command:
`pg_dump -n edbuser -f /pgbackup/edbuser_schema.sql -U postgres edbstore` then type the **postgres** password.
3. Take a data-only dump of the `edbstore` database, disable all triggers for a faster restore, and use the `insert` command instead of `copy`, then name the file as **edbstore_data.sql**.

Type the following command:

```
pg_dump -a --inserts --disable-triggers -f  
/pgbackup/edbstore_data.sql -U postgres edbstore then type  
the postgres password.
```

4. Take a full dump of `customers` table and name the file **edbuser_customers.sql**.
Type the following command:
`pg_dump -t edbuser.customers -f
/pgbackup/edbuser_customers.sql -U postgres edbstore` then
type the **postgres** password.

Lab Exercise - 3

1. Take a full database dump of `edbstore` in compressed format using the **pg_dump** utility, name the file as **edbstore_full_fc.dmp**



2. Take a full database cluster dump using **pg_dumpall**. Remember **pg_dumpall** supports only plain text format; name the file **edbdata.sql**

Answer Lab Exercise - 3

Open a terminal and login as the postgres user. Type
`su - postgres` then the password.

1. Take a full database dump of the `edbstore` in compressed format using the **pg_dump** utility, name the file as **edbstore_full_fc.dmp**.
Type the following command:
`pg_dump -Fc -f /pgbackup/edbstore_full_fc.dmp -U postgres edbstore` then type the **postgres** password.
2. Take a full database cluster dump using **pg_dumpall**. Remember **pg_dumpall** supports only plain text format; name the file **edbdata.sql**.
Type the following command:
`pg_dumpall -p 5432 -U postgres > /pgbackup/edbdata.sql`
then type the **postgres** password 5-6 times as required.

Lab Exercise - 4

In this exercise you will demonstrate your ability to restore a database.

1. Drop database `edbstore`.
2. Create database `edbstore` with owner `edbuser`.
3. Restore the full dump from **edbstore_full.sql** and verify all the objects and their ownership.
4. Drop database `edbstore`.
5. Create database `edbstore` with `edbuser` owner.
6. Restore the full dump from the compressed file **edbstore_full_fc.dmp** and verify all the objects and their ownership.

Answer Lab Exercise - 4

Open a terminal and login as the postgres user.
Type
`su - postgres` then the password.

1. Drop database `edbstore`.
Type the following command:
`dropdb edbstore` then type the **postgres** password
2. Create database `edbstore` with owner `edbuser`.
Type the following command:
`createdb -O edbuser edbstore` then type the **postgres** password.

List the back up files



Type the following command: `ls /pgbackup`

```
[postgres@TrainingServer ~]$ ls /pgbackup
edbdata.sql  edbstore_data.sql  edbstore_full_fc.dmp  edbstore_full.sql  edbuser_customers.sql  edbuser_schema.sql
```

3. Restore the full dump from **edbstore_full.sql** and verify all the objects and their ownership.

Type the following command:

```
psql -f /pgbackup/edbstore_full.sql -d edbstore -U
edbuser then type the password for edbuser.
```

- a. Go to the edbstore database as the edbuser user.

Type

```
psql -d edbstore -U edbuser then type the edbuser
password
```

- b. List the tables of edbstore. Type `\dt`

```
edbstore=> \dt
          List of relations
Schema | Name          | Type  | Owner
-----+-----+-----+-----
edbuser | categories    | table | edbuser
edbuser | cust_hist     | table | edbuser
edbuser | customers     | table | edbuser
edbuser | dept          | table | edbuser
edbuser | emp           | table | edbuser
edbuser | emp2          | table | edbuser
edbuser | inventory     | table | edbuser
edbuser | job_grd       | table | edbuser
edbuser | jobhist       | table | edbuser
edbuser | locations     | table | edbuser
edbuser | orderlines    | table | edbuser
edbuser | orders        | table | edbuser
edbuser | products      | table | edbuser
edbuser | reorder       | table | edbuser
(14 rows)
```

Exit out of psql Type `\q`

4. Drop database edbstore.

Type the following command:

```
dropdb edbstore then type the password for postgres.
```

5. Create database edbstore with edbuser owner.

Type the following command:

```
createdb -O edbuser edbstore then type the password.
```

6. Restore the full dump from compressed file **edbstore_full_fc.dmp** and verify all the objects and their ownership.

Type the following command:

```
pg_restore -d edbstore /pgbackup/edbstore_full_fc.dmp
```

Login to psql for the edbstore database.



Type the following command:

```
psql -d edbstore -U edbuser
```

List the edbstore tables type `\dt`

```
edbstore=> \dt
```

List of relations			
Schema	Name	Type	Owner
edbuser	categories	table	edbuser
edbuser	cust_hist	table	edbuser
edbuser	customers	table	edbuser
edbuser	dept	table	edbuser
edbuser	emp	table	edbuser
edbuser	emp2	table	edbuser
edbuser	inventory	table	edbuser
edbuser	job_grd	table	edbuser
edbuser	jobhist	table	edbuser
edbuser	locations	table	edbuser
edbuser	orderlines	table	edbuser
edbuser	orders	table	edbuser
edbuser	products	table	edbuser
edbuser	reorder	table	edbuser

(14 rows)

Exit out of psql Type `\q`

Lab Exercise - 5

1. Create a directory `/opt/arch` or `c:\arch` and give ownership to the `postgres` user.
2. Configure your cluster to run in archive mode and set the archive log location to be `/opt/arch` or `c:\arch`.
3. Take a full online base backup of your cluster in the `/pgbackup` directory using the **pg_basebackup** utility.

Answer Lab Exercise - 5

1. Open a terminal and logon as the root user.
Type
`su - root` then type the root password.
 - a. Create a directory `/opt/arch` or `c:\arch` and give ownership to Postgres user.
Type `mkdir /opt/arch`
 - b. Give ownership to Postgres user
Type `chown postgres:postgres /opt/arch/`
2. Connect as the user postgres.
Type
`su - postgres`



- a. Open the config file.

```
vi /var/lib/pgsql/16/data/postgresql.conf
```

- b. Change following parameters. Type <i> to get into insert mode

```
wal_level = replica
archive_mode = on
archive_command = 'cp %p /opt/arch/%f'
max_wal_senders = 2
```

- c. Save and close the vi editor. Type <ESC>:wq <ENTER>

- d. Open the config file.

Type

```
vi /var/lib/pgsql/16/data/pg_hba.conf
```

- e. **Uncomment following entries and change authentication method to scram-sha-256:**

```
local      replication postgres                                scram-
                                                    sha-256
host       replication postgres 127.0.0.1/32                  scram-
                                                    sha-256
host       replication postgres ::1/128                      scram-
                                                    sha-256
```

- f. Save and close the vi editor.

Type<Esc>:wq <ENTER>

- g. Restart the server.

Type the following command:

```
pg_ctl -D /var/lib/pgsql/16/data -mf restart
```

3. Take the backup.

Type the following command:

```
pg_basebackup -h localhost -D /pgbackup/data
```



Module 13 - Routine Maintenance Tasks

Lab Exercise – 1

1. While monitoring table statistics on the **edbstore** database, you found that some tables are not automatically maintained by **autovacuum**. You decided to perform manual maintenance on these tables. Write a SQL script to perform the following:
 - Reclaim obsolete row space from the `customers` table.
 - Update statistics for `emp` and `dept` tables.
 - Mark all the obsolete rows in the `orders` table for reuse.
2. Execute the newly created maintenance script on **edbstore** database.

Answer Lab Exercise - 1

Open a terminal and login as the postgres user. Type
`su - postgres` then the **postgres** password.

1. Write a SQL script to perform maintenance on the `customers`, `emp`, `dept` and `orders` tables.

Type the following command:

`vi edbstore_mnts_script.sql` to create the script.

Then type `<i>` to get into insert mode and enter the following maintenance commands:

```
VACUUM FULL customers;  
ANALYZE emp;  
ANALYZE dept;  
VACUUM orders;
```

Save and close. Type

`<ESC>:wq <ENTER>`

2. Run the script.

Type the following command:

`psql -f edbstore_mnts_script.sql -d edbstore -U edbuser`
then enter the **edbuser** password.

```
[postgres@pgcentos1 ~]$ psql -f edbstore_mnts_script.sql -d edbstore -U edbuser  
VACUUM  
ANALYZE  
ANALYZE  
VACUUM
```



Lab Exercise - 2

1. The composite index named `ix_orderlines_orderid` on (`orderid`, `orderlineid`) columns of the `orderlines` table is performing very slow. Write a statement to reindex this index for better performance.

Answer Lab Exercise - 2

Open a terminal and login as the postgres user. Type `su - postgres` then the password.

Go to psql in the edbstore database. Type `psql edbstore edbuser` and type the **edbuser** password

1. Write the reindex statement.
Type the following command:
`REINDEX INDEX ix_orderlines_orderid;`

Exit out of psql type `\q`



Module 14 - Moving Data

Lab Exercise - 1

1. Unload the `emp` table from the `edbuser` schema to a csv file, with column headers.
2. Create a `copyemp` table with same structure as the `emp` table.
3. Load the csv file (from step 1) into the `copyemp` table.

Answer Lab Exercise - 1

Open a terminal and login as the postgres user. Type `su - postgres` and type the password.

Go to psql in the edbstore database. Type `psql edbstore postgres` and type the **postgres** password.

1. Unload `emp` table from `edbuser` schema to a csv file, with column headers.
Type the following command:

```
COPY edbuser.emp to '/home/postgres/emp.csv' with csv
header;
```

```
edbstore=# copy edbuser.emp to '/home/postgres/empc.csv' with csv header;
COPY 14
```

2. Create a `copyemp` table with same structure as `emp` table.
Type the following command:

```
CREATE TABLE copyemp (like edbuser.emp);
```

3. Load the csv file(from step 1) into `copyemp` table.
Type the following command:

```
COPY copyemp FROM '/home/postgres/emp.csv' with csv
header;
```

```
edbstore=# create table copyemp (like edbuser.emp);
CREATE TABLE
edbstore=# copy copyemp from '/home/postgres/emp.csv' with csv header;
COPY 14
```

Review the data in the `copyemp` table. Type `SELECT * FROM copyemp;`

Exit psql type `\q`